

CSCI 232

232 Algorithms & Data Structures

LECTURE: 16

J. L. Pach

TODAY'S AGENDA

- Insertion Sort
  - The idea, a trace, the code
  - Loop invariant, $T(n)$, $S(n)$, stability
- Sorting Our Own Objects
  - Class `Square` and the overloaded `operator<`
  - One sorting function for `int` and for `Square`
- Polymorphism
  - Abstract class `Shape`, derived classes
  - Base pointer, reference, copy (slicing)
  - Why `*a < *b` and not `a < b`

INSERTION

Sort

THE IDEA: CARDS IN YOUR HAND

- You hold a **sorted** hand and take the next card from the table.
- You slide it to the left, past every bigger card, until it fits.
- The sorted part grows by one card in every step.

```
sorted part | unsorted part
 2  5  |  4  1        take 4: slide it left past 5
 2  4  5  |  1        take 1: slide it left past 5, 4, 2
 1  2  4  5  |
```

Trace on {5, 2, 4, 1}

Step	key	Array after the step
start		5 2 4 1
i = 1	2	2 5 4 1
i = 2	4	2 4 5 1
i = 3	1	1 2 4 5

> Elements bigger than `key` move **one cell to the right**. `key` is written into the gap that opens.

INSERTION SORT IN C++

```
void insertionSort(int A[], int n)
{
    int key; int j;
    for (int i = 1; i < n; i++)
    {
        key = A[i];                 // the card we insert
        j = i - 1;
        while (j >= 0 && A[j] > key)      // slide bigger elements right
        {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;                 // drop the card into the gap
    }
}
```

Invariant, T(n), S(n)

- **Loop invariant:** before iteration i, the cells `A[0..i-1]` hold the **same elements** as at the start, in sorted order.
- **Best case** (already sorted): the `while` never runs, $T(n) = \Theta(n)$.
- **Worst case** (reversed): the `while` runs i times for every i, so $T(n) = 1 + 2 + \cdots + (n-1) = \Theta(n^2)$.
- **Space:** `key`, `i`, `j`, so $S(n) = \Theta(1)$: the sort is **in place**.
- **Stable:** equal elements keep their order, because we move only when `A[j] > key` (strictly).

> Compare with bubble sort: the same $\Theta(n^2)$ worst case, but insertion sort is $\Theta(n)$ on sorted input and does fewer writes.

SORTING OUR

Own Objects

PROBLEM: SORTING SQUARES

```
class Square
{
   private: double side;
   public:  explicit Square(double s) : side(s) {}
   public:  double area() const { return side * side; }
};

Square a(3), b(2);
if (a < b) { }       // error: no match for 'operator<'
```



- For `int` the compiler knows what `<` means. For our class it does **not**.
- Our sort needs only one question: *"is this element smaller than that one?"*
- We answer it by **overloading** `operator<`.

Overloading operator `<`

```cpp
class Square {
   private: double side;
   public:  explicit Square(double s) : side(s) {}
   public:  double area() const { return side * side; }

   public:  bool operator < (const Square& other) const
   {
      return area() < other.area();      // we decide: compare by area
   }
};

Square a(3), b(2);
a < b;       // the compiler rewrites it as a.operator<(b)  ->  false
```



- `const Square&`: we only read `other` and avoid a copy. The trailing `const`: we do not modify `*this`.
- `operator <` is an **ordinary method** with a special name.

One Sort for int and for Square

```
template <typename T>
void insertionSort(T A[], int n)
{
   for (int i = 1; i < n; i++)
   {
      T key = A[i];
      int j = i - 1;
      while (j >= 0 && key < A[j])      // the ONLY operation on T besides copying
      {
         A[j + 1] = A[j];
         j--;
      }
      A[j + 1] = key;
   }
}

int    x[] = {5, 2, 4, 1};        insertionSort(x, 4);   // 1 2 4 5
Square s[] = {Square(3), Square(1), Square(2)};
insertionSort(s, 3);              // sorted by area: 1, 2, 3
```

WHY ONLY OPERATOR `<`?

- We wrote `key < A[j]` instead of `A[j] > key`. Then the algorithm needs **one** operator.
- The same `operator <` is enough for `std::sort`, `std::set`, `std::map`, `std::max`, ...
- It must be a **strict weak order**: `a < a` is false, and if `a < b` and `b < c` then `a < c`.
- Two squares with equal area: `a < b` and `b < a` are both false. Insertion sort leaves them in place (stable).

> Rule of thumb: **never** use `<=` as your `operator <`. A sort with `<=` is not stable and can run off the array.

POLYMORPHISM

Problem: Squares, Circles, Rectangles

- An array holds elements of **one type**. How do we sort squares, circles and rectangles **together**?
- Idea: all of them are **shapes** and all can answer `area()`.
- We describe "a shape" once, in an **abstract class**. The classes derived from it must supply the details.

```
class Shape {
   public: virtual ~Shape() = default;          // see the last slides
   public: virtual double area() const = 0;     // "= 0": pure virtual, no body

   public: bool operator<(const Shape& other) const
   {
      return area() < other.area();             // calls the DERIVED area()
   }
};
```



Derived Classes

```cpp
class Circle : public Shape {
    private: double r;
    public:  explicit Circle(double r) : r(r) {}
    public:  double area() const override { return std::numbers::pi * r * r; }
};
class Rectangle : public Shape {
    private: double w, h;
    public:  Rectangle(double w, double h) : w(w), h(h) {}
    public:  double area() const override { return w * h; }
};
class Square : public Shape {
    private: double side;
    public:  explicit Square(double s) : side(s) {}
    public:  double area() const override { return side * side; }
};

// Shape s;   // error: cannot declare a variable of abstract type
```

`override` makes the compiler check that we really replace a virtual method of the base class.

1. POINTER TO THE BASE CLASS

```
Shape* arr[3] = { new Circle(1), new Square(2), new Rectangle(1, 8) };

for (Shape* s : arr)
    std::cout << s->area() << "\n";     // 3.14159, 4, 8: three different area()

for (Shape* s : arr) delete s;
```



- `Circle*` converts to `Shape*` **implicitly**. Every circle *is* a shape.
- **Nothing is cut off.** The object stays whole in memory. Only our *view* of it is narrower.
- `s->area()` is resolved **at run time** from the real type. This is **polymorphism**.

HOW DOES IT WORK? (C ANALOGY)

In C you did this by hand:

```c
struct Shape
{
    double (*area)(struct Shape*);
    /* data ... */
};
s->area(s);
```



In C++ the compiler does it for you:

- a class with `virtual` methods gets a hidden pointer (**vptr**) to a table of its methods,
- `Circle` fills the table with `Circle::area`,
- `s->area()` follows vptr, so the real type decides.

> The information about the real type is **stored in the object**, so a base pointer loses nothing.

2. REFERENCE TO AN EXISTING OBJECT

```
void describe(const Shape& s)              // works for ANY shape
{
   std::cout << "area = " << s.area() << "\n";
}

Circle c(1);
Square q(2);
describe(c);          // area = 3.14159
describe(q);          // area = 4

Shape& r = c;         // a reference to the base: also polymorphic
r.area();           // Circle::area
```



A reference is the best parameter type for **one** object. It cannot be `nullptr`, and it needs no `new` and no `delete`.

3. Copy: Slicing

```
class Animal { public: virtual std::string sound() const { return "..."; } };
class Dog : public Animal { public: std::string sound() const override { return "Woof"; } };

Dog d;
Animal* p = &d;     p->sound();     // "Woof"   pointer: OK
Animal& r = d;      r.sound();      // "Woof"   reference: OK
Animal  a = d;      a.sound();      // "..."    COPY: slicing!
```



- `Animal a = d;` creates a **new, smaller** `Animal` and copies only the base part. The `Dog` part and its vptr are gone.
- Polymorphism needs a **pointer or reference**. A **value** of the base type is always just the base.
- Our `Shape` is abstract, so `Shape s = c;` does not even compile: the compiler protects us.

CONTAINERS: WHY POINTERS

```cpp
std::list<Shape>  a;     // error: Shape is abstract
std::list<Shape*> b;     // OK: elements are addresses

b.push_back(new Circle(1));
b.push_back(new Square(2));
b.push_back(new Rectangle(1, 4));
```



- Container cells have **one fixed size**, but a `Circle` and a `Rectangle` have different sizes.
- A pointer has the same size for every shape, and the object itself stays whole on the heap.
- Python and Java do the same silently: every variable there already holds a reference.
- Whoever puts `new` shapes into the list **owns** them and must `delete` them.

THE TRAP: COMPARING POINTERS

```
Shape* a = new Circle(1);       // area 3.14
Shape* b = new Square(1);       // area 1

bool wrong = a < b;             // compares ADDRESSES, like char* in C
bool right = *a < *b;           // compares objects: Shape::operator< -> area()
```



- `a` and `b` are pointers, so `a < b` is the built-in pointer comparison. Our overloaded `operator<` is **never called**.
- The result is "random": it depends on where `new` put the objects.
- **Always dereference:** `*a < *b`, or `**it < **jt` for list iterators.

> It compiles, it runs, it gives wrong answers. The most common bug in this topic.

The Same Template, Silently Wrong

```
Shape* arr[3] = { new Circle(1), new Square(2), new Rectangle(1, 4) };
insertionSort(arr, 3);      // T = Shape*  ->  key < A[j] compares ADDRESSES
```



The template from before compiles for `T = Shape*`, but `key < A[j]` now compares addresses. We write a version for pointers:

```
void insertionSort(Shape* A[], int n){
    for (int i = 1; i < n; i++)
    {
        Shape* key = A[i];
        int j = i - 1;
        while (j >= 0 && *key < *A[j])        // dereference both sides
        {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
}
```



VIRTUAL DESTRUCTOR

```
Shape* p = new Circle(1);
delete p;                // which destructor runs?
```



- Without `virtual ~Shape()`, `delete p` runs only `Shape`'s destructor: the `Circle` part is never destroyed (**undefined behavior**, leaks if `Circle` owns memory).
- With `virtual ~Shape() = default;` the destructor of the **real type** runs first.
- **Rule:** a class meant to be a base class gets a **virtual destructor**.

Do and Don't

Do	Don't
`Shape*` or `const Shape&` to use shapes polymorphically	`Shape s = circle;` (slicing)
`*a < *b` for pointers	`a < b` for pointers (addresses)
`virtual ~Shape()` in a base class	`delete` through a base pointer without it
`override` on every replaced method	silently hiding a method with another signature
one owner calls `delete` exactly once	`delete` the same object twice
`const Shape&` for a read-only parameter	pass `Shape` by value

SUMMARY

- **Insertion sort**: $\Theta(n)$ best, $\Theta(n^2)$ worst, $S(n) = \Theta(1)$, stable.
- A sort needs only `operator<`. Overload it, and the same algorithm sorts `int` and your own classes.
- **Abstract class** `Shape` with `virtual area() = 0`: derived classes supply the details.
- Polymorphism works through a **base pointer or reference**. A **copy** of the base type is sliced.
- Pointers must be **dereferenced** before `<`, and a base class needs a **virtual destructor**.

THANK

You